

坐标

2026年3月10日 17:56

Maven坐标

- 什么是坐标?
 - Maven 中的坐标是资源(jar)的唯一标识, 通过该坐标可以唯一定位资源位置。
 - 使用坐标来定义项目或引入项目中需要的依赖。
- Maven 坐标主要组成
 - groupId: 定义当前Maven项目隶属组织名称(通常是域名反写, 例如: com.itheima)
 - artifactId: 定义当前Maven项目名称(通常是模块名称, 例如 order-service、goods-service)
 - version: 定义当前项目版本号
 - SNAPSHOT: 功能不稳定、尚处于开发中的版本, 即快照版本
 - RELEASE: 功能趋于稳定、当前更新停止, 可以用于发行的版本



```
pom.xml
<groupId>com.itheima</groupId>
<artifactId>maven-project01</artifactId>
<version>1.0-SNAPSHOT</version>

pom.xml
<dependency>
  <groupId>cn.hutool</groupId>
  <artifactId>hutool-all</artifactId>
  <version>5.8.27</version>
</dependency>
```

添加依赖

2026年3月10日 19:39

依赖配置



- 依赖：指当前项目运行所需要的jar包，一个项目中可以引入多个依赖。

- 配置：

1. 在 pom.xml 中编写 `<dependencies>` 标签
2. 在 `<dependencies>` 标签中 使用 `<dependency>` 引入坐标
3. 定义坐标的 `groupId`, `artifactId`, `version`
4. 点击刷新按钮，引入最新加入的坐标

```
pom.xml
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.4</version>
</dependency>
```



注意：

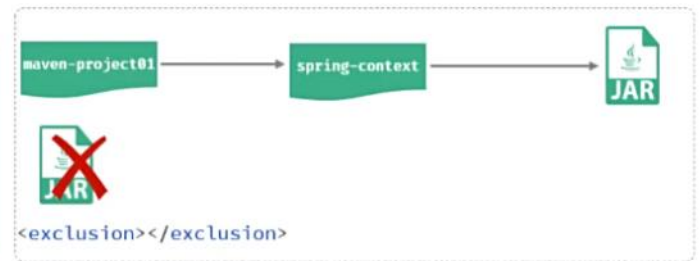
- 如果不知道依赖的坐标信息，可以到 <https://mvnrepository.com/> 中搜索。

排除依赖

- 排除依赖：指主动断开依赖的资源，被排除的资源**无需指定版本**。

```
pom.xml<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.4</version>

  <exclusions>
    <exclusion>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-observation</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```



生命周期

2026年3月10日 19:43

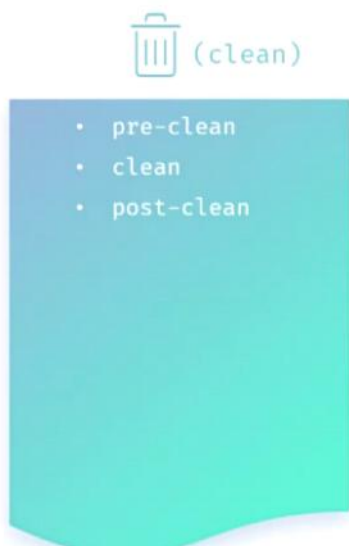
Maven的生命周期就是为了对所有的maven项目构建过程进行抽象和统一。



Maven中有3套**相互独立**的生命周期：

- clean：清理工作。
- default：核心工作，如：编译、测试、打包、安装、部署等。
- site：生成报告、发布站点等。

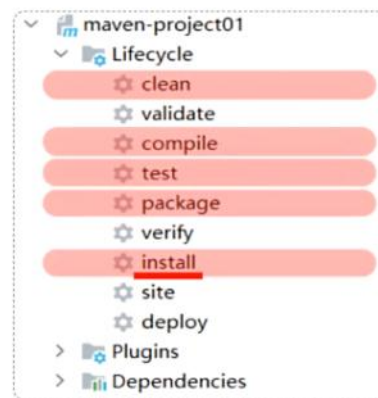
每套生命周期包含一些阶段（phase），阶段是有顺序的，后面的阶段依赖于前面的阶段。



生命周期阶段

- **clean**: 移除上一次构建生成的文件
- **compile**: 编译项目源代码
- **test**: 使用合适的单元测试框架运行测试(junit)
- **package**: 将编译后的文件打包, 如: jar, war等
- **install**: 安装项目到本地仓库

51%



注意:

在同一套生命周期中, 当运行后面的阶段时, 前面的阶段都会运行。

依赖范围

2026年3月12日 16:01

- 依赖的jar包，默认情况下，可以在任何地方使用。可以通过 `<scope>...</scope>` 设置其作用范围。
- 作用范围：
 - 主程序范围有效。（main文件夹范围内）
 - 测试程序范围有效。（test文件夹范围内）
 - 是否参与打包运行。（package指令范围内）



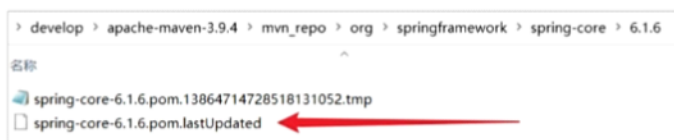
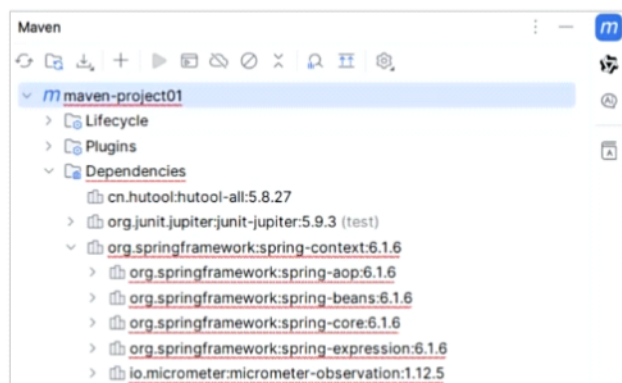
```
pom.xml
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.9.3</version>
  <scope>test</scope>
</dependency>
```

scope值	主程序	测试程序	打包（运行）	范例
compile（默认）	Y	Y	Y	log4j
test	-	Y	-	junit
provided	Y	Y	-	servlet-api
runtime	-	Y	Y	jdbc驱动

依赖下载

2026年3月12日 16:01

Maven常见问题解决方案



- 产生原因：由于网络原因，依赖没有下载完整导致的，在maven仓库中生成了xxx.lastUpdated文件，该文件不删除，不会再重新下载。
- 解决方案：
 1. 根据maven依赖的坐标，找到仓库中对应的 xxx.lastUpdated 文件，删除，删除之后重新加载项目即可。
 2. 通过命令 (`del /s *.lastUpdated`) 批量递归删除指定目录下的 xxx.lastUpdated 文件，删除之后重新加载项目即可。



注意：

重新加载依赖，依赖下载了之后，maven面板可能还会报红，此时可以关闭IDEA，重新打开IDEA加载此项目即可。